

Lernstruktur

Kategorie	Thema	Detaillierte Beschreibung	Lernziele
Grundlagen	Lombok	Automatisierte Code-Generierung	Lombok ist eine Java-Bibliothek, die Boilerplate-Code reduziert, indem sie Annotationen zur automatischen Generierung von Getter-, Setter-, Konstruktor- und Logging-Methoden bereitstellt.
	Class Base	Eine Klasse ist die fundamentale Struktureinheit in Java. Sie stellt den Bauplan für Objekte dar und kapselt Attribute (Felder) sowie Verhalten (Methoden). Eine Klasse dient als Vorlage für die Erstellung von Instanzen.	Verständnis der objektorientierten Prinzipien, Kenntnis über die Struktur und den Aufbau einer Klasse, Fähigkeit zur Modellierung realer Konzepte in Java.
	Class Static	<code>static</code> wird verwendet, um Methoden und Variablen zu definieren, die auf Klassenebene existieren und nicht an eine bestimmte Instanz gebunden sind. Statische Elemente werden oft für Konstanten oder Hilfsmethoden genutzt.	Verständnis der Unterschiede zwischen Instanz- und Klassenmitgliedern, Fähigkeit zur sinnvollen Nutzung statischer Variablen und Methoden.
	Interface	Ein Interface definiert eine Menge von Methoden, die eine implementierende Klasse bereitstellen muss. Es ermöglicht eine lose Kopplung, fördert Wiederverwendbarkeit und erleichtert das Arbeiten mit Polymorphismus.	Fähigkeit, Schnittstellen zur Abstraktion und Strukturierung von Code zu verwenden, Verständnis für lose Kopplung und Modularisierung.
	Generics	Generics ermöglichen eine typsichere Programmierung durch Platzhalter für Datentypen. Sie verhindern Laufzeitfehler durch fehlerhafte Typkonvertierungen und verbessern die Wiederverwendbarkeit von Code.	Verständnis der Notwendigkeit und Vorteile von Generics, sichere Anwendung generischer Klassen, Methoden und Wildcards (<code>?</code> , <code>extends</code> , <code>super</code>).
Kommandozeile	String... args	Die Übergabe von Kommandozeilenargumenten erlaubt die flexible Konfiguration eines Programms zur Laufzeit. Diese Argumente werden als Array in die <code>main</code> -Methode übergeben.	Verständnis für die Handhabung von Kommandozeilenparametern, Fähigkeit zur Verarbeitung von Benutzerargumenten beim Start eines Programms.
Konfiguration	Enum	Enums definieren eine feste Menge von Konstanten und bieten eine typsichere Möglichkeit, Zustände oder Konfigurationswerte darzustellen. Sie verbessern die Lesbarkeit und Wartbarkeit des Codes.	Fähigkeit, Enums zur Repräsentation von fixen Wertebereichen zu nutzen, Verständnis für deren Anwendung in Konfigurationen und Statusverwaltung.
Kontrollstrukturen	If	Die <code>if</code> -Bedingung erlaubt das bedingte Ausführen von Codeblöcken basierend auf booleschen Ausdrücken. Sie ist eine der grundlegenden Steuerungsstrukturen in Java.	Fähigkeit zur Implementierung von Entscheidungsstrukturen, Verständnis für bedingte Verzweigungen im Programmfluss.

Kategorie	Thema	Detaillierte Beschreibung	Lernziele
	If-Else	Die <code>if-else</code> -Struktur erweitert <code>if</code> , indem eine alternative Anweisung ausgeführt wird, falls die Bedingung nicht erfüllt ist. Dadurch können Programme verschiedene Pfade einschlagen.	Verständnis für alternative Abläufe innerhalb von Programmen, Fähigkeit zur Strukturierung komplexer Entscheidungsprozesse.
	Switch	Die <code>switch</code> -Anweisung ermöglicht die effiziente Mehrfachverzweigung basierend auf festen Werten. Sie eignet sich für klare und strukturierte Auswahlentscheidungen.	Fähigkeit zur Nutzung von <code>switch</code> für übersichtliche und effiziente Mehrfachverzweigungen, Verständnis für dessen Einsatzgebiete und Einschränkungen.
Schleifen	For	Die <code>for</code> -Schleife ermöglicht eine kontrollierte Wiederholung eines Codeblocks, oft mit einem Zähler. Sie wird häufig für Iterationen über bekannte Zahlenbereiche verwendet.	Verständnis für das Prinzip der Iteration, Fähigkeit zur effektiven Nutzung von <code>for</code> -Schleifen zur Automatisierung wiederholender Prozesse.
	While	Die <code>while</code> -Schleife wiederholt Anweisungen solange eine Bedingung erfüllt ist. Sie ist besonders nützlich für Prozesse mit unbekannter Wiederholungsanzahl.	Fähigkeit zur Implementierung bedingungsgesteuerter Schleifen, Verständnis für das Zusammenspiel von Bedingung und Iteration.
Datentypen	Boolean	Der <code>boolean</code> -Typ speichert Wahrheitswerte (<code>true</code> oder <code>false</code>). Er wird für logische Vergleiche und Steuerungsstrukturen verwendet.	Verständnis der logischen Werte in Java, Fähigkeit zur Nutzung von <code>boolean</code> für Entscheidungsfindungen.
	Char	Der <code>char</code> -Typ speichert ein einzelnes Zeichen basierend auf Unicode. Er wird für Zeichenoperationen und Texterstellungen genutzt.	Verständnis des Unicode-Zeichensatzes, Fähigkeit zur Manipulation einzelner Zeichen.
	Byte, Short, Int, Long	Diese Typen repräsentieren Ganzzahlen mit unterschiedlicher Speichergröße und Wertebereichen. Sie werden für numerische Berechnungen und Datenverarbeitung genutzt.	Verständnis der Unterschiede zwischen den numerischen Ganzzahltypen, Fähigkeit zur Wahl des passenden Datentyps für verschiedene Anwendungsfälle.
	Float, Double	Gleitkommazahlen mit einfacher (<code>float</code>) und doppelter (<code>double</code>) Genauigkeit zur Darstellung von Dezimalzahlen.	Verständnis für die Genauigkeit und Präzision von Gleitkommazahlen, Fähigkeit zur Nutzung für Berechnungen mit Fließkommazahlen.
	String	<code>String</code> speichert und verarbeitet Zeichenketten, die aus einer Sequenz von <code>char</code> -Werten bestehen.	Fähigkeit zur Verarbeitung von Zeichenketten, Verständnis der String-Methoden zur Manipulation von Textdaten.
	LocalDate	<code>LocalDate</code> repräsentiert ein Datum ohne Zeit- und Zeitoneninformationen im ISO-8601-Kalendersystem.	Fähigkeit zur Verarbeitung von Datumswerten, Verständnis der Methoden zur Manipulation und Formatierung von Datumseinträgen.
	Array	Arrays sind Datenstrukturen, die mehrere Elemente desselben Typs enthalten und über Indizes zugreifbar sind.	Verständnis für die Handhabung statischer Datenstrukturen, Fähigkeit zur effizienten Nutzung von Arrays zur Speicherung und Verarbeitung von Daten.

Kategorie	Thema	Detaillierte Beschreibung	Lernziele
Sammlungen & Datenstrukturen	ArrayList	Eine dynamische Liste, die ihre Größe automatisch anpasst. Sie ermöglicht flexible Datenverwaltung ohne feste Begrenzung.	Verständnis für die dynamische Speicherverwaltung, Fähigkeit zur Nutzung von <code>ArrayList</code> für flexible Datenspeicherung.
	HashMap	Eine Schlüssel-Wert-Speicherstruktur, die eine effiziente Suche und Zuordnung von Elementen erlaubt.	Verständnis für die effiziente Speicherung von Zuordnungen, Fähigkeit zur Nutzung von <code>HashMap</code> zur schnellen Datenabfrage.
	Queue & Stack	Eine Warteschlange (Queue) verwendet FIFO (First-In-First-Out), ein Stack arbeitet nach LIFO (Last-In-First-Out).	Verständnis für die Konzepte von Stapel- und Warteschlangen-Datenstrukturen, Fähigkeit zur Nutzung in Algorithmen und Problemstellungen.
Beziehungen zwischen Klassen	Association	Eine Assoziation beschreibt eine lose Beziehung zwischen zwei Klassen, bei der Objekte miteinander in Verbindung stehen, aber voneinander unabhängig existieren können.	Verständnis für lose gekoppelte Objektbeziehungen und deren Anwendung in Modellierungsprozessen.
	Aggregation	Eine Aggregation ist eine schwache Form der Komposition, bei der eine Klasse eine andere Klasse enthält, aber das enthaltene Objekt unabhängig existiert.	Fähigkeit zur Identifikation und Implementierung von Aggregationen zur Wiederverwendung von Objekten.
	Composition	Die Komposition ist eine starke Form der Aggregation, bei der das enthaltene Objekt ohne die umschließende Klasse nicht existieren kann.	Verständnis für enge Objektbeziehungen und deren Anwendung in Modellen, in denen eine strikte Zugehörigkeit notwendig ist.
	Inheritance	Die Vererbung erlaubt es einer Klasse, Eigenschaften und Methoden einer anderen Klasse zu übernehmen. Sie ist eine der Kernkonzepte der objektorientierten Programmierung.	Verständnis der Vererbungshierarchie, Fähigkeit zur Nutzung von <code>extends</code> , Basisklassen und abgeleiteten Klassen.
Methoden	Overloading	Beim Method Overloading werden mehrere Methoden mit demselben Namen, aber unterschiedlichen Parametertypen oder -anzahlen definiert.	Fähigkeit zur Implementierung überladener Methoden zur Verbesserung der Code-Wiederverwendbarkeit.
	Overriding	Beim Method Overriding überschreibt eine Unterklasse eine Methode der Oberklasse, um spezifisches Verhalten zu definieren.	Verständnis der Unterschiede zwischen Overloading und Overriding, Fähigkeit zur Implementierung polymorpher Methoden.
Datenstrukturen & Algorithmen	TreeMap	Eine Map-Implementierung, die Elemente in einer sortierten Reihenfolge speichert.	Fähigkeit, <code>TreeMap</code> zur Speicherung und Abfrage von Daten in sortierter Reihenfolge zu nutzen.
	Sorting	Comparator	Ein <code>Comparator</code> wird verwendet, um benutzerdefinierte Sortierlogik zu definieren, die über die natürliche Reihenfolge hinausgeht.
	Search	Die Implementierung von Algorithmen zur effizienten Suche von Elementen, wie binäre Suche oder lineare Suche.	Verständnis für die Implementierung und Optimierung von Suchalgorithmen.
Ein- und Ausgabe	Eingabe	Scanner	Der <code>Scanner</code> wird verwendet, um Eingabedaten von der Konsole oder anderen Quellen zu lesen.

Kategorie	Thema	Detaillierte Beschreibung	Lernziele
	BufferedReader	Ein effizienter Pufferleser, der zum schnellen Einlesen von Zeichenströmen verwendet wird.	Verständnis der Nutzung von <code>BufferedReader</code> zur Optimierung des Lesevorgangs von Zeichenströmen.
	BufferedWriter	Ein effizienter Puffer-Schreiber, der zum schnellen Schreiben von Zeichenströmen verwendet wird.	Fähigkeit zur effizienten Ausgabe von Zeichenströmen mit <code>BufferedWriter</code> .
Identität & Vergleiche	equals	Die <code>equals</code> -Methode wird verwendet, um Objekte in Java auf ihre inhaltliche Gleichheit zu vergleichen.	Verständnis der Funktionsweise und Bedeutung der <code>equals</code> -Methode bei der Objektvergleichen.
Lambda / Streams	Funktionale Verarbeitung von Daten	Lambdas und Streams ermöglichen eine deklarative und funktionale Verarbeitung von Datenmengen, indem sie Operationen wie Filterung, Transformation und Reduktion unterstützen.	Fähigkeit zur Anwendung von Lambdas und Streams zur effizienten und eleganten Verarbeitung von Daten, einschließlich Filterung, Mapping und Reduzierung.
Kryptografie	SHA, AES, RSA	Diese Algorithmen dienen der sicheren Datenverschlüsselung und Hashing. Sie werden für Passwortspeicherung, Datenverschlüsselung und sichere Kommunikation verwendet.	Verständnis der kryptografischen Grundprinzipien, Fähigkeit zur Wahl und Anwendung passender Algorithmen zur Datensicherheit.